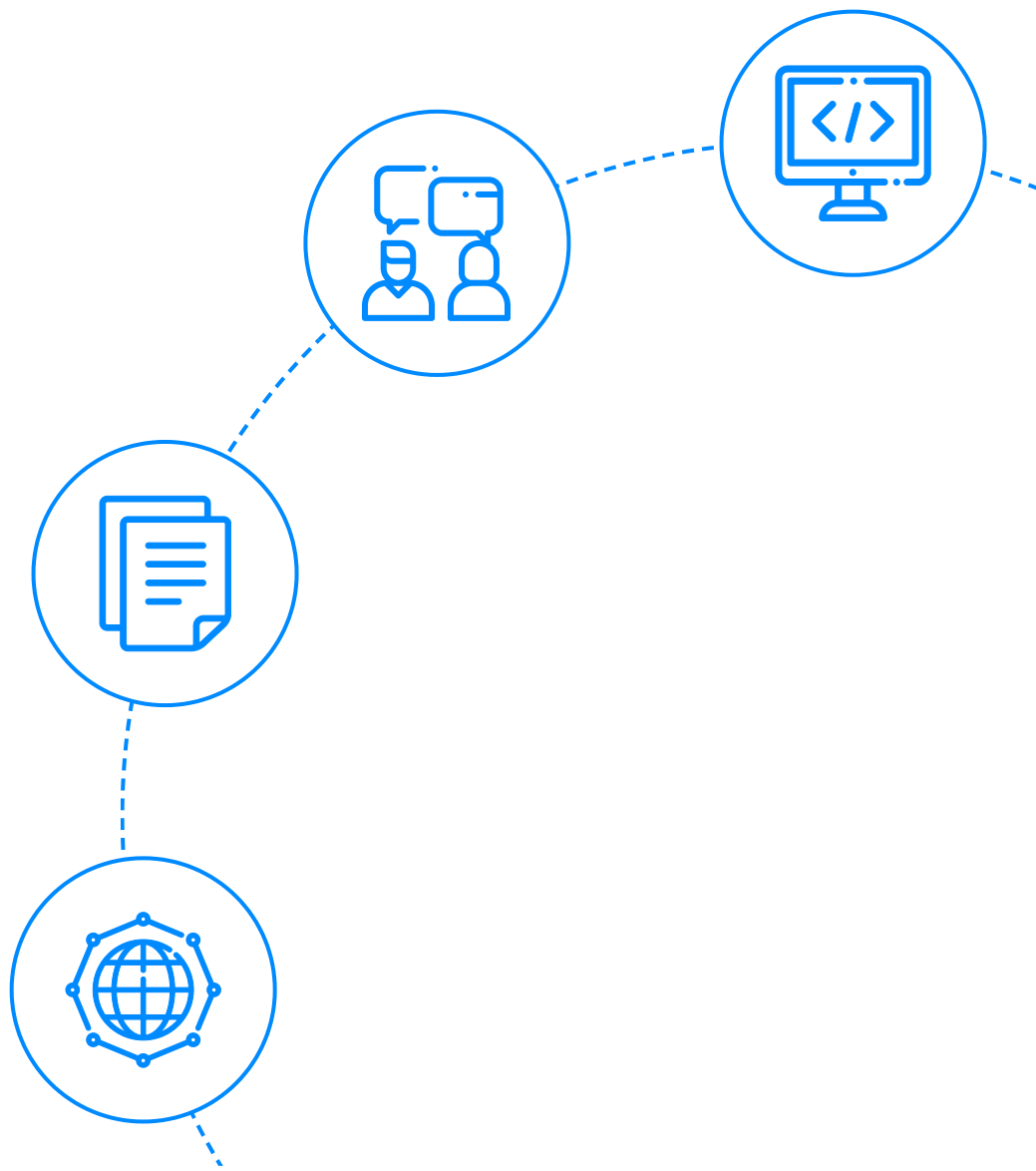




InterviewBit

Generative AI Interview Questions and Answers You Must Prepare for 2026



To view the live version of the page, [click here](#).

© Copyright by Interviewbit

Contents

Evaluation, Hallucination & Safety

1. How do you implement content filtering and guardrails for LLM outputs?
2. What are common security vulnerabilities in LLM applications?
3. What is LLM-as-a-Judge evaluation?
4. What causes hallucinations in LLMs and how do you detect and reduce them?
5. How do you evaluate the quality of LLM outputs? Explain BLEU, ROUGE, and BERTScore.

Fine-Tuning and Training

6. How do you prepare datasets for fine-tuning LLMs?
7. What is instruction tuning and why is it important?
8. What is RLHF (Reinforcement Learning from Human Feedback)? Explain the pipeline.
9. Explain LoRA (Low-Rank Adaptation) and QLoRA - how do they reduce compute requirements?
10. When should you fine-tune an LLM vs use RAG vs prompt engineering?

LLM Fundamentals

11. What is generative AI and how does it differ from discriminative AI?
12. What are the key limitations of current LLMs (hallucinations, context limits, knowledge cutoff)?
13. What is tokenization? Explain BPE, WordPiece, and SentencePiece.
14. What is the difference between encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5) architectures?
15. Explain the transformer architecture and self-attention mechanism.

Production Deployment & System Design

Production Deployment & System Design (.....Continued)

- 16. Design a document Q&A system using RAG - walk through the architecture.
- 17. What is LangChain? Explain its core components.
- 18. Explain model quantization (INT8, INT4) and its trade-offs.
- 19. What are the key challenges in deploying LLMs to production?

Prompt Engineering

- 20. How do you design prompts to reduce hallucinations?
- 21. What is ReAct (Reasoning + Acting) prompting?
- 22. How do temperature and top-p (nucleus sampling) affect LLM outputs?
- 23. What is chain-of-thought (CoT) prompting and when should you use it?
- 24. Explain zero-shot, one-shot, and few-shot prompting with examples.

Retrieval-Augmented Generation (RAG)

- 25. How do you evaluate RAG system performance (faithfulness, relevance, answer correctness)?
- 26. What is chunking and what strategies exist (fixed, semantic, recursive)?
- 27. What are vector databases and how do you choose between Pinecone, Weaviate, Chroma, and FAISS?
- 28. What is the difference between sparse retrieval (BM25) and dense retrieval?
- 29. Explain the components of a RAG pipeline (retriever + generator).
- 30. What is RAG and what problem does it solve?

Let's get Started

Generative AI interviews in 2026 are getting trickier, and we completely understand what most students go through. You can be asked ANYTHING, and the mere fear of it sometimes tattles on your confidence, but worry not! As we have prepared these Questions and Answers, just so you have an idea about what kind of questions are asked in the interviews and what kind of answers they expect.

Interviewers usually expect you to understand how large language models work, why certain design choices are made, and when to use techniques like prompt engineering, RAG, or fine-tuning in actuality.

We've curated these 30 most-asked generative AI interview questions based on current hiring trends across startups and large tech companies. Whether you're a fresher stepping into AI, a software engineer transitioning to GenAI, or an experienced ML practitioner preparing for senior roles, these questions will help you gain clarity over the concepts and help you prepare for your interviews.

Evaluation, Hallucination & Safety

1. How do you implement content filtering and guardrails for LLM outputs?

Content filtering and guardrails are mechanisms used to constrain LLM behavior and prevent unsafe, biased, or policy-violating outputs. These controls are typically enforced at multiple layers of the system.

Guardrail implementation strategies

- 1. Input Filtering:** Block or rewrite harmful or malicious user prompts
- 2. Output Filtering:** Scan generated text for policy violations before delivery

3. Prompt-Level Constraints: Embed safety rules and refusal conditions in system prompts

4. Rule-Based Enforcement: Apply deterministic rules for restricted content

5. Model-Based Moderation: Use classification models to flag unsafe outputs

6. Human-in-the-Loop: Escalate high-risk cases for manual review

Effective guardrail design helps in balancing safety, usability, and performance, which makes it a critical component of production-grade LLM systems.

2. What are common security vulnerabilities in LLM applications?

LLM-based systems introduce new security risks because they process untrusted user input and generate executable or authoritative outputs.

Common vulnerabilities

1. Prompt Injection

- Malicious instructions override system prompts
- Can lead to data leakage or policy bypass

2. Jailbreaking: Crafted inputs force models to ignore safety constraints

3. Data Leakage: Model exposes sensitive or proprietary information

4. Tool Misuse: Models invoke tools or APIs in unintended ways

5. Indirect Prompt Attacks: Malicious content embedded in retrieved documents

Securing LLM systems requires treating prompts, retrieval data, and tool outputs as untrusted inputs.

3. What is LLM-as-a-Judge evaluation?

LLM-as-a-Judge is an evaluation technique where a language model is used to assess the quality of another model's output. Instead of relying only on automated metrics or human reviewers, an LLM scores responses based on predefined criteria.

This approach is especially useful for subjective tasks such as reasoning quality, helpfulness, and factual grounding.

How it works

- Define evaluation criteria (accuracy, relevance, faithfulness, safety)
- Provide the generated output and reference context to the judge model
- Ask the model to score or rank responses
- Aggregate scores across multiple samples

Advantages

- Scales better than an evaluation done by a person
- Enables rapid iteration during development
- Useful for comparing multiple model versions

Limitations

- Judge model bias
- Sensitivity to evaluation prompt design
- Requires careful calibration and validation

4. What causes hallucinations in LLMs and how do you detect and reduce them?

Hallucinations occur when LLMs generate outputs that are not grounded in training data, provided context, or verified external sources. This behavior arises from how language models optimize for likelihood rather than factual correctness.

Primary causes

- 1. Lack of grounding:** Prompts do not provide sufficient context or reference data
- 2. Knowledge gaps:** Queries fall outside training data or exceed the model's knowledge cutoff
- 3. Ambiguous or underspecified prompts:** Model fills gaps with statistically plausible text
- 4. Overgeneralization:** Model extrapolates patterns beyond supported evidence

Detection and reduction strategies

- 1. Retrieval grounding:** Use RAG to anchor responses in retrieved documents
- 2. Prompt constraints:** Enforce strict instruction boundaries and source limitations
- 3. Structured reasoning:** Require step-by-step or evidence-based answers
- 4. Output verification:** Use post-generation validation or secondary models
- 5. Confidence calibration:** Detect unsupported claims using heuristics or LLM-based judges

Hallucination control is a system-level problem that requires coordinated design across prompting, retrieval, and evaluation layers.

5. How do you evaluate the quality of LLM outputs? Explain BLEU, ROUGE, and BERTScore.

Evaluating LLM outputs is challenging because many tasks do not have a single correct answer. As a result, evaluation often combines automated metrics with human or model-based judgment.

1. BLEU (Bilingual Evaluation Understudy)

- Measures n-gram overlap between generated text and reference text
- Precision-focused metric
- Commonly used in machine translation
- Performs poorly for open-ended generation

2. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- Measures overlap between generated and reference text
- Recall-focused metric
- Widely used for summarization tasks
- Does not capture semantic similarity well

3. BERTScore

- Uses contextual embeddings to measure semantic similarity
- Captures meaning rather than exact word overlap
- Better suited for modern generative tasks
- More computationally expensive than BLEU or ROUGE

In practice, these metrics are task-dependent and are often supplemented with evaluations taken by a person in charge or LLM-based evaluators.

Fine-Tuning and Training

6. How do you prepare datasets for fine-tuning LLMs?

Dataset preparation directly impacts fine-tuning performance because LLMs are highly sensitive to data quality, formatting, and diversity. Poor dataset construction can introduce bias, reduce generalization, or degrade pretrained knowledge.

Dataset preparation focuses on aligning training samples with target task objectives.

Key dataset preparation steps

1. Define Task Objectives: Identify output format, domain scope, and performance goals

2. Data Cleaning and Normalization

- Remove duplicates, noise, and irrelevant samples
- Standardize text formatting and token structure

3. Prompt-Response Structuring

- Convert raw data into instruction-based training pairs
- Maintain consistency across dataset examples

4. Data Diversity and Balance

- Include varied examples to improve generalization
- Avoid overrepresenting specific data categories

5. Context Window Optimization

- Ensure samples fit within token limits
- Preserve meaningful context without truncation

6. Dataset Splitting

- Separate training, validation, and evaluation sets
- Monitor overfitting and performance drift

High-quality dataset preparation often contributes more to fine-tuning success which is why it is frequently discussed in LLM system design interviews.

7. What is instruction tuning and why is it important?

Instruction tuning is a supervised training approach where LLMs are trained using datasets containing natural language instructions paired with expected outputs. The objective is to teach models how to interpret user queries across multiple tasks using structured instruction patterns.

Instruction tuning improves model generalization and reduces the complexity required in prompt design.

Key benefits

- 1. Task Adaptability:** Enables models to handle multiple tasks using natural language instructions
- 2. Response Consistency:** Improves adherence to formatting and output constraints
- 3. Reduced Prompt Complexity:** Minimizes the need for long or example-heavy prompts
- 4. Multi-Task Performance:** Allows models to generalize across translation, summarization, reasoning, and classification tasks

Instruction tuning is often the step that transforms pretrained models into interactive AI assistants.

8. What is RLHF (Reinforcement Learning from Human Feedback)? Explain the pipeline.

RLHF aligns LLM behavior with human expectations by optimizing outputs based on preference feedback.

The RLHF training workflow typically involves multiple stages that progressively refine response quality.

Pipeline stages

1. Pretraining

- Train the base language model on large-scale text corpora
- Learn grammar, reasoning patterns, and general knowledge

2. Supervised Fine-Tuning (SFT)

- Train using curated prompt-response pairs created by human annotators
- Improve instruction-following behavior

3. Reward Model Training

- Train a ranking model using human preference comparisons
- Assign quality scores to alternative model outputs

4. Reinforcement Learning Optimization

- Optimize the LLM using reward model feedback
- Commonly implemented using Proximal Policy Optimization (PPO)

RLHF improves helpfulness, safety, and alignment, which makes it central to conversational AI training.

9. Explain LoRA (Low-Rank Adaptation) and QLoRA - how do they reduce compute requirements?

LoRA and QLoRA are parameter-efficient fine-tuning methods designed to reduce memory and computational costs while adapting large language models.

1. LoRA (Low-Rank Adaptation)

- Freezes pretrained model weights
- Introduces small trainable matrices using low-rank decomposition
- Reduces the number of parameters that require training
- Enables multiple task-specific adapters without duplicating base models

2. QLoRA (Quantized LoRA)

- Applies quantization to compress model weights into lower precision formats
- Retains training stability while reducing GPU memory requirements
- Allows fine-tuning of large models using limited hardware resources

These techniques enable scalable model customization while maintaining base model performance.

10. When should you fine-tune an LLM vs use RAG vs prompt engineering?

Fine-tuning, RAG, and prompt engineering solve different categories of LLM limitations. The choice depends on whether the requirement involves behavior control, knowledge retrieval, or output formatting.

1. Prompt Engineering

- Controls response tone, format, and reasoning pattern
- Requires no retraining or infrastructure changes
- Useful for quick experimentation and instruction clarity

2. Retrieval-Augmented Generation (RAG)

- Injects external or real-time knowledge during inference
- Suitable for enterprise documents and frequently updated data
- Improves factual grounding and reduces hallucinations

3. Fine-Tuning

- Modifies model weights using domain-specific datasets
- Improves consistency and specialized language understanding
- Effective for workflow automation and organization-specific use cases

LLM Fundamentals

11. What is generative AI and how does it differ from discriminative AI?

Generative AI refers to models designed to create new data that resembles the training data. These models learn patterns, structures, and relationships in data and then use that knowledge to generate text, images, audio, code, or other content.

Examples include:

- Chatbots generating human-like responses
- AI tools creating images from text prompts
- Code assistants generating programming solutions

Generative AI models learn the joint probability distribution of input data, which allows them to produce entirely new outputs.

Discriminative AI, on the other hand, focuses on classification or prediction tasks. Instead of generating new content, these models learn to distinguish between categories or predict labels.

Examples include:

- Spam detection models classifying emails
- Image recognition systems identifying objects
- Sentiment analysis models detecting emotions in text

Key Differences

Aspect	Generative AI	Discriminative AI
Purpose	Creates new content	Classifies or predicts
Learning Focus	Learns full data distribution	Learns decision boundaries
Output Type	Text, images, audio, code	Categories or probabilities
Example Models	GPT, Stable Diffusion	Logistic Regression, SVM

In interviews, candidates are often expected to explain that generative models focus on creation, while discriminative models focus on classification or prediction.

12. What are the key limitations of current LLMs (hallucinations, context limits, knowledge cutoff)?

Despite rapid advancements, LLMs still face several technical and practical limitations.

1. Hallucinations

LLMs sometimes generate incorrect or fabricated information that appears factually accurate.

Why It Happens:

- Models predict likely text rather than verifying facts
- Training data inconsistencies
- Lack of real-time knowledge validation

2. Context Window Limits

LLMs can only process a limited number of tokens at once. Information outside this window is ignored.

Impact:

- Difficulty handling long documents
- Loss of earlier conversation details
- Performance degradation in complex reasoning tasks

3. Knowledge Cutoff

LLMs are trained on data available up to a certain point in time and may not include recent developments.

Impact:

- Outdated information
- Limited awareness of current events
- Requires external retrieval systems like RAG for updates

Additional Practical Limitations

- High computational and infrastructure costs
- Sensitivity to prompt phrasing
- Potential bias from training data
- Security vulnerabilities like prompt injection

Understanding these limitations is crucial because interviewers often evaluate whether candidates know when LLMs should not be used or require additional safeguards.

If you're preparing for generative AI interview questions and answers, getting well-versed with these LLM fundamentals will help you confidently tackle advanced topics such as prompt engineering, RAG pipelines, and production deployment.

13. What is tokenization? Explain BPE, WordPiece, and SentencePiece.

Tokenization is the process of breaking text into smaller units called tokens. Tokens can be words, subwords, or characters, depending on the tokenization method used.

LLMs cannot process raw text directly, so tokenization converts text into numerical representations that models can understand.

1. Byte Pair Encoding (BPE)

BPE starts with individual characters and merges frequently occurring character combinations to form subwords.

Advantages:

- Handles rare words efficiently
- Reduces vocabulary size
- Maintains balance between word-level and character-level tokens

Example:

“unhappiness” -> “un” + “happi” + “ness”

2. WordPiece

WordPiece is similar to BPE but selects token merges based on probability improvements rather than frequency alone.

Advantages:

- Provides better language representation
- Commonly used in encoder-based models
- Produces consistent subword tokens

3. SentencePiece

SentencePiece treats text as a raw stream of characters without requiring pre-tokenization based on spaces.

Advantages:

- Language-independent
- Works well for multilingual models
- Handles languages without clear word boundaries

Tokenization directly impacts model efficiency, context length, and performance, which makes it a common topic in LLM basics interview discussions.

14. What is the difference between encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5) architectures?

LLMs are generally categorized based on how they process input and generate output.

1. Encoder-Only Models (BERT)

Encoder-only architectures focus on understanding input text rather than generating new text.

Key Characteristics:

- Processes entire input simultaneously
- Produces contextual embeddings
- Used mainly for classification and understanding tasks

Common Use Cases:

- Sentiment analysis
- Named entity recognition
- Question answering

2. Decoder-Only Models (GPT)

Decoder-only models specialize in text generation by predicting the next token in a sequence.

Key Characteristics:

- Uses causal attention (looks only at previous tokens)
- Generates text step-by-step
- Strong performance in conversational AI and content generation

Common Use Cases:

- Chatbots
- Code generation
- Content writing

3. Encoder-Decoder Models (T5)

Encoder-decoder models combine both understanding and generation capabilities.

Key Characteristics:

- Encoder processes input text
- Decoder generates output text
- Suitable for transformation tasks

Common Use Cases:

- Translation
- Summarization
- Paraphrasing

Quick Comparison

Architecture	Primary Function	Example Model
Encoder-Only	Text understanding	BERT
Decoder-Only	Text generation	GPT
Encoder-Decoder	Input-output transformation	T5

15. Explain the transformer architecture and self-attention mechanism.

The transformer architecture is an important part of LLMs and is widely asked in transformer interview questions. Introduced in 2017, transformers replaced older sequential models like RNNs and LSTMs by allowing models to process entire sequences simultaneously.

A transformer primarily consists of:

- Input embeddings
- Positional encoding
- Attention layers
- Feed-forward neural networks
- Normalization and residual connections

Self-Attention Mechanism

Self-attention allows a model to determine how important each word in a sentence is relative to other words. Instead of processing text sequentially, the model evaluates relationships between all tokens at once.

For example, in the sentence: “The dog chased the ball because it was fast.”

Self-attention helps the model determine whether “it” refers to the dog or the ball by analyzing contextual relationships.

The self-attention mechanism works using three key components:

- **Query (Q)** - Represents the token being evaluated
- **Key (K)** - Represents tokens being compared
- **Value (V)** - Contains contextual information

The model calculates similarity between queries and keys to assign attention scores, which are then applied to values to generate context-aware representations.

Why Transformers Became Popular

- Capture long-range dependencies effectively
- Enable parallel processing
- Scale efficiently for large datasets
- Provide better contextual understanding

Most large language models you’ll encounter today rely on this architecture, which is why it appears so often in generative AI interview discussions.

Production Deployment & System Design

16. Design a document Q&A system using RAG - walk through the architecture.

A document Q&A system using RAG enables users to query large document collections and receive grounded answers derived from stored knowledge sources. The system integrates retrieval mechanisms with LLM-based generation to ensure factual and context-aware responses.

Architecture workflow

1. Data Ingestion

- Collect and preprocess documents
- Perform text cleaning and normalization

2. Chunking and Embedding

- Split documents into smaller segments
- Convert text chunks into vector embeddings

3. Vector Storage

- Store embeddings in a vector database
- Enable similarity-based retrieval

4. Query Processing

- Convert user query into embedding representation
- Retrieve top relevant document chunks

5. Prompt Construction

- Combine user query with retrieved context
- Apply structured prompt templates

6. Response Generation: LLM generates answers grounded in retrieved content

7. Post-Processing and Evaluation

- Apply safety filters and output validation
- Log responses for monitoring and performance evaluation

17. What is LangChain? Explain its core components.

LangChain is a framework designed to simplify development of applications powered by LLMs. It provides structured abstractions that help developers connect language models with external data sources, tools, and workflows.

LangChain enables modular construction of multi-step LLM pipelines and agent-based systems.

Core components

1. Chains

- Define sequential workflows combining prompts, models, and data processing steps.
- Used for structured task execution

2. Agents

- Enable dynamic decision making
- Allow LLMs to choose tools or actions based on user queries

3. Memory

- Stores conversation or session context
- Enables stateful interactions and personalized responses

4. Tools

- Provide interfaces to external APIs, databases, or computational functions
- Allow LLMs to perform real-world operations

LangChain is commonly used in RAG pipelines, conversational assistants, and agent-driven automation systems.

18. Explain model quantization (INT8, INT4) and its trade-offs.

Model quantization is an optimization technique that reduces memory and computational requirements by converting model weights and activations into lower-precision numerical formats.

Quantization enables faster inference and reduced hardware usage while maintaining acceptable model performance.

1. INT8 Quantization

- Converts weights from 32-bit floating point to 8-bit integers
- Reduces memory footprint and improves inference speed
- Maintains strong performance for most language tasks

2. INT4 Quantization

- Further compresses weights into 4-bit precision
- Enables deployment on memory-constrained hardware
- May introduce noticeable accuracy degradation

Trade-offs

Accuracy vs Efficiency

- Lower precision reduces resource usage
- Higher compression may reduce model output quality

Hardware Compatibility

- Some accelerators are optimized for INT8 workloads
- Extreme compression requires specialized inference frameworks

Quantization is widely used in production environments to reduce infrastructure costs while maintaining acceptable performance levels.

19. What are the key challenges in deploying LLMs to production?

Production LLM deployment involves balancing performance, cost efficiency, and system reliability while maintaining response quality. Since LLMs require high computational resources and interact with multiple system components, deployment complexity increases significantly compared to traditional ML models.

Major production challenges

1. Latency

- Large models increase response generation time
- Multi-step pipelines, such as RAG or tool calls, add processing overhead
- Requires caching, batching, and optimized inference strategies

2. Cost

- High GPU and infrastructure requirements
- Token-based pricing increases operational expenses
- Requires model optimization and request routing strategies

3. Scalability

- Must support concurrent user requests
- Requires load balancing and distributed inference
- Infrastructure must adapt to variable traffic loads

4. Reliability

- Requires fallback mechanisms and monitoring pipelines
- Must handle model failures or degraded responses

5. Safety and Compliance

- Requires moderation layers and guardrail enforcement
- Must prevent data leakage and misuse

Production-ready LLM systems typically combine optimized inference infrastructure, monitoring pipelines, and layered safety controls.

Prompt Engineering

20. How do you design prompts to reduce hallucinations?

Hallucinations occur when LLMs generate outputs that are not supported by training data, provided context, or external sources. Since LLMs operate by predicting the most probable next token rather than validating factual correctness, hallucinations often arise when prompts are ambiguous, underspecified, or lack grounding information.

From a system design perspective, reducing hallucinations through prompt engineering involves constraining the model's generation space and enforcing context-dependent reasoning. Well-structured prompts explicitly define task boundaries, input sources, and response formats, which reduces the likelihood of unsupported generation.

A key strategy is instructing the model to rely strictly on supplied context or retrieved documents. When models are forced to generate answers only from provided inputs, the probability of fabricating information drops significantly. Limiting output scope further discourages speculative completion.

Effective hallucination reduction techniques:

- Clearly specify response boundaries. Example: Answer only using the provided document.
- Provide contextual information or reference data
- Encourage step-by-step reasoning
- Restrict output format or length
- Ask the model to verify or cite sources
- Combine prompting with retrieval systems like RAG

Strong prompt design is often the fastest way to improve the reliability of the model without retraining or fine-tuning LLMs, which makes it a critical skill evaluated in generative AI interviews.

21. What is ReAct (Reasoning + Acting) prompting?

ReAct (Reasoning + Acting) prompting is a technique where an LLM interleaves explicit reasoning steps with external actions while solving a task. Instead of generating an answer solely from its internal knowledge, the model reasons about what information it needs, performs actions such as querying tools or APIs, and then incorporates the retrieved results into its final response.

Example workflow:

- User asks: *What is the population of Canada's capital city?*
- Model reasoning: Identify Canada's capital city
- Action: Retrieve information about Ottawa
- Model reasoning: Extract population data
- Final answer: Provide verified population details

Key advantages of ReAct prompting:

- Enables tool-augmented reasoning
- Improves factual accuracy and traceability
- Supports multi-step decision making
- Forms the basis of agent-based AI systems

ReAct prompting is widely used in applications such as autonomous agents, enterprise search, customer support bots, and workflow automation, which is why it frequently appears in advanced prompt engineering interview questions.

22. How do temperature and top-p (nucleus sampling) affect LLM outputs?

Temperature and top-p are decoding parameters that influence how probability distributions are used when an LLM predicts the next token. Rather than always selecting the highest probability token, these parameters modify how candidate tokens are sampled, which directly affects response diversity and determinism.

Temperature rescales the probability distribution of possible next tokens. Lower temperature values concentrate probability around the most likely tokens, resulting in safer and more consistent outputs. Higher temperature values spread probability more evenly across tokens, allowing the model to explore alternative phrasing and creative responses.

Examples:

Low temperature (0.1 - 0.3)

- Produces deterministic and factual responses
- Useful for technical documentation or coding

Medium temperature (0.5 - 0.7)

- Balances creativity and accuracy
- Common in chatbot responses

High temperature (0.8 - 1.0+)

- Generates imaginative and varied content
- Suitable for storytelling or brainstorming

Top-p sampling limits token selection to a subset of words whose combined probability reaches a defined threshold. This ensures the model chooses from highly likely tokens while maintaining diversity.

Examples:

Top-p = 0.9

- Model selects tokens from the most probable 90% of outcomes
- Produces natural but controlled responses

In interviews, you should explain that temperature influences randomness globally, while top-p controls probability-based token selection.

23. What is chain-of-thought (CoT) prompting and when should you use it?

Chain-of-thought prompting encourages LLMs to explain intermediate reasoning steps before producing a final answer. Instead of jumping directly to a result, the model breaks the problem into smaller logical stages, which improves accuracy and maintains transparency.

This technique is especially useful when tasks involve calculations, logical deductions, or multi-step decision-making. It also helps developers verify whether the model's reasoning process is valid.

Example:

Standard Prompt: If 3 books cost \$30, how much do 6 books cost?

Output: \$60

Chain-of-Thought Prompt: Solve step-by-step: If 3 books cost \$30, how much do 6 books cost?

Output:

- Cost per book = \$10
- Cost of 6 books = \$60

Common use cases:

- Mathematical and analytical reasoning
- Coding and debugging
- Complex decision workflows
- Multi-hop question answering

24. Explain zero-shot, one-shot, and few-shot prompting with examples.

Zero-shot, one-shot, and few-shot prompting describe how many task examples are given to an LLM before requesting an output. These techniques help control accuracy, formatting consistency, and reasoning quality.

Zero-shot prompting involves giving the model only instructions, without any examples. The model depends entirely on knowledge learned during training. It works well for simple or commonly learned tasks but may struggle with domain-specific queries.

Example:

- Prompt: *Classify the sentiment of the text: "The product quality is amazing."*
- Output: Positive

One-shot prompting provides a single example to demonstrate the expected response style or logic. This helps the model understand formatting and improves reliability compared to zero-shot prompting.

Example:

Prompt:

- Text: "The service was terrible." - Negative
- Text: "The delivery was fast and smooth." -?

Output: Positive

Few-shot prompting includes multiple examples, allowing the model to learn patterns more effectively. This is commonly used for structured outputs, domain-specific tasks, and higher accuracy requirements.

Example:

Prompt:

- Hello - Bonjour
- Thank you - Merci
- Good night -?

Output: Bonne nuit

Retrieval-Augmented Generation (RAG)

25. How do you evaluate RAG system performance (faithfulness, relevance, answer correctness)?

Evaluating a RAG system requires assessing both retrieval quality and generation quality. Unlike standard LLM evaluation, RAG evaluation focuses on whether answers are grounded in retrieved data.

Key evaluation dimensions:

- Faithfulness: Measures whether generated answers are supported by retrieved documents
- Relevance: Evaluates whether retrieved content aligns with the user query
- Answer correctness: Assesses factual accuracy and completeness of the response

Additional evaluation techniques include:

- Comparing answers against ground-truth references
- Using LLM-as-a-judge for qualitative scoring
- Tracking retrieval precision and recall

In interviews, you will be expected to explain that RAG evaluation is system-level, and it requires monitoring, retrieval, prompting, and generation together.

26. What is chunking and what strategies exist (fixed, semantic, recursive)?

Chunking is the process of splitting large documents into smaller segments before embedding and indexing them for retrieval. Proper chunking is critical because LLMs have context window limits and retrieval quality depends on chunk relevance.

Different chunking strategies balance context preservation and retrieval precision.

Common chunking strategies:

Fixed-size chunking

- Splits text into equal-length chunks
- Simple but may break semantic boundaries

Semantic chunking

- Splits text based on meaning or topic shifts
- Preserves contextual coherence

Recursive chunking

- Uses hierarchical rules (paragraph - sentence - clause)
- Balances structure and flexibility

Note: Poor chunking often leads to irrelevant retrievals, even if embeddings and vector databases are well configured.

27. What are vector databases and how do you choose between Pinecone, Weaviate, Chroma, and FAISS?

Vector databases are specialized systems designed to store, index, and search high-dimensional vector embeddings efficiently. In RAG pipelines, they are used to retrieve semantically similar documents based on embedding distance.

Choosing a vector database depends on factors such as scale, deployment requirements, and system complexity.

Key selection considerations:

Pinecone

- Fully managed, production-ready
- Strong scalability and reliability
- Suitable for large-scale enterprise systems

Weaviate

- Open-source with managed options
- Supports hybrid search and metadata filtering
- Good for flexible, schema-aware applications

Chroma

- Lightweight and developer-friendly
- Ideal for local development and prototyping
- Often used with LangChain-based pipelines

FAISS

- High-performance similarity search library
- Requires custom infrastructure setup
- Common in research and custom deployments

28. What is the difference between sparse retrieval (BM25) and dense retrieval?

Sparse and dense retrieval differ in how text is represented and matched during search.

Sparse retrieval methods, such as BM25, rely on exact or near-exact term matching between queries and documents. They use statistical term-frequency techniques to score relevance.

Dense retrieval represents queries and documents as dense vector embeddings. Semantic similarity is computed using vector distance, allowing retrieval based on meaning rather than exact wording.

Key differences:

Sparse retrieval (BM25):

- Uses keyword-based matching
- Performs well for exact queries
- Lightweight and easy to implement

Dense retrieval:

- Uses embedding similarity
- Handles paraphrasing and semantic variation
- Better suited for conversational and natural language queries

In practice, many production RAG systems use hybrid retrieval, combining both sparse and dense approaches to maximize recall and precision.

29. Explain the components of a RAG pipeline (retriever + generator).

A RAG system is typically composed of two core components that work sequentially: a retriever and a generator.

The **retriever** is responsible for identifying the most relevant documents or text chunks based on a user query. It converts the query into a searchable representation and retrieves relevant information from a knowledge store.

The **generator** is an LLM that consumes both the user query and the retrieved content to generate a final answer. The model is instructed to ground its response strictly in the retrieved context.

Typical RAG pipeline flow:

- User query is received
- Query is embedded or transformed for retrieval
- Relevant documents or chunks are fetched
- Retrieved content is injected into the prompt
- LLM generates a grounded response

This separation allows retrieval logic and generation logic to be optimized independently.

30. What is RAG and what problem does it solve?

RAG is an approach that combines information retrieval with text generation to produce grounded and context-aware responses. Instead of asking an LLM to answer purely from memory, the system first retrieves relevant documents or data and then uses those results as context for generation.

The primary problem RAG solves is that LLMs do not verify facts and cannot access up-to-date or private information by default. By injecting retrieved knowledge into the prompt, RAG reduces hallucinations and enables models to answer queries using current or domain-specific data.

Key problems RAG addresses:

- Hallucinations caused by unsupported generation
- Knowledge cutoff limitations
- Inability to access private or proprietary data
- Poor performance on domain-specific queries

Links to More Interview Questions

[Accenture Interview Questions](#)

[Accolite Interview Questions](#)

[Adobe Interview Questions](#)

[Amazon Interview Questions](#)

[Amdocs Interview Questions](#)

[Apisero Interview Questions](#)

[Arcesium Interview Questions](#)

[Avasoft Interview Questions](#)

[Axis Bank Interview Questions](#)

[Capgemini Interview Questions](#)

[Cisco Interview Questions](#)

[Cognizant Interview Questions](#)

[Dassault Systemes Interview Questions](#)

[De Shaw Interview Questions](#)

[Delhivery Interview Questions](#)

[Deloitte Interview Questions](#)

[Directi Interview Questions](#)

[Dxc Interview Questions](#)

[Epam Interview Questions](#)

[Ey Interview Questions](#)

[Facebook Interview Questions](#)

[Flipkart Interview Questions](#)

[Genc Interview Questions](#)

[Genpact Interview Questions](#)

[Goldman Sachs Interview Questions](#)

[Google Interview Questions](#)

[Hashedin Interview Questions](#)

[Hcl Interview Questions](#)

[Hexaware Interview Questions](#)

[Ibm Interview Questions](#)

[Infosys Interview Questions](#)

[Intuit Interview Questions](#)

[Jp Morgan Interview Questions](#)